Neural Networks

- Lecture 1 -

Introduction, Supervised Learning, Backpropagation

The Team

Instructor: Alexandru Sorici

(alexandru.sorici@upb.ro)



Project / HW assistents team:

Teodor Poncu (dan_teodor.poncu@upb.ro)



Vlad Florea (vlad.florea1709@upb.ro)



Andrei Dugăeșescu (andrei.dugaesescu@upb.ro)



Cătălin Chiru (catalin.chiru@upb.ro)



Outline

- Housekeeping :-)
- What is this class about?
- Recap on Supervised Learning
- Backpropagation

Housekeeping – Semester Organization

- **14** lectures
- **3** Programming and Analysis Homework Assignments (**30p**)
- **1** project in 3 steps (**60p**)
  - Step 1: Topic selection and SotA presentation
    - Presentation + paper (Introduction and Related Work sections)
  - Step 2: Intermediary project evaluation (e.g. first results) – **30p**
    - Presentation + continued paper (Method description and first results)
  - Step 3: Final Project Poster Presentation (**counts as part of exam**) – **30p**
    - Poster + final paper (Final experiments, Final results, Conclusion)
- **Project Hours** to be used as Office Hours + Short Tutorial Coverage
- **1** written final exam (**10p**)
- **Passing conditions:**
  - **50% of semester activity required for exam entry (**grades for HW + Step 2 of project)
  - **50% of final project**

Housekeeping – Syllabus

- See lecture and practical work schedule link

Suggested Reading Material

- Books:
  - Ian Goodfellow, Yoshua Bengio, Aaron Courville Deep Learning Book
  - Christopher Bishop, Hugh Bishop
    Deep Learning - Foundations and Concepts
  - Deisenroth, A. Aldo Faisal, Cheng Soon Ong:
    Mathematics for Machine Learning

- Tutorials:
  - To be covered during project hours

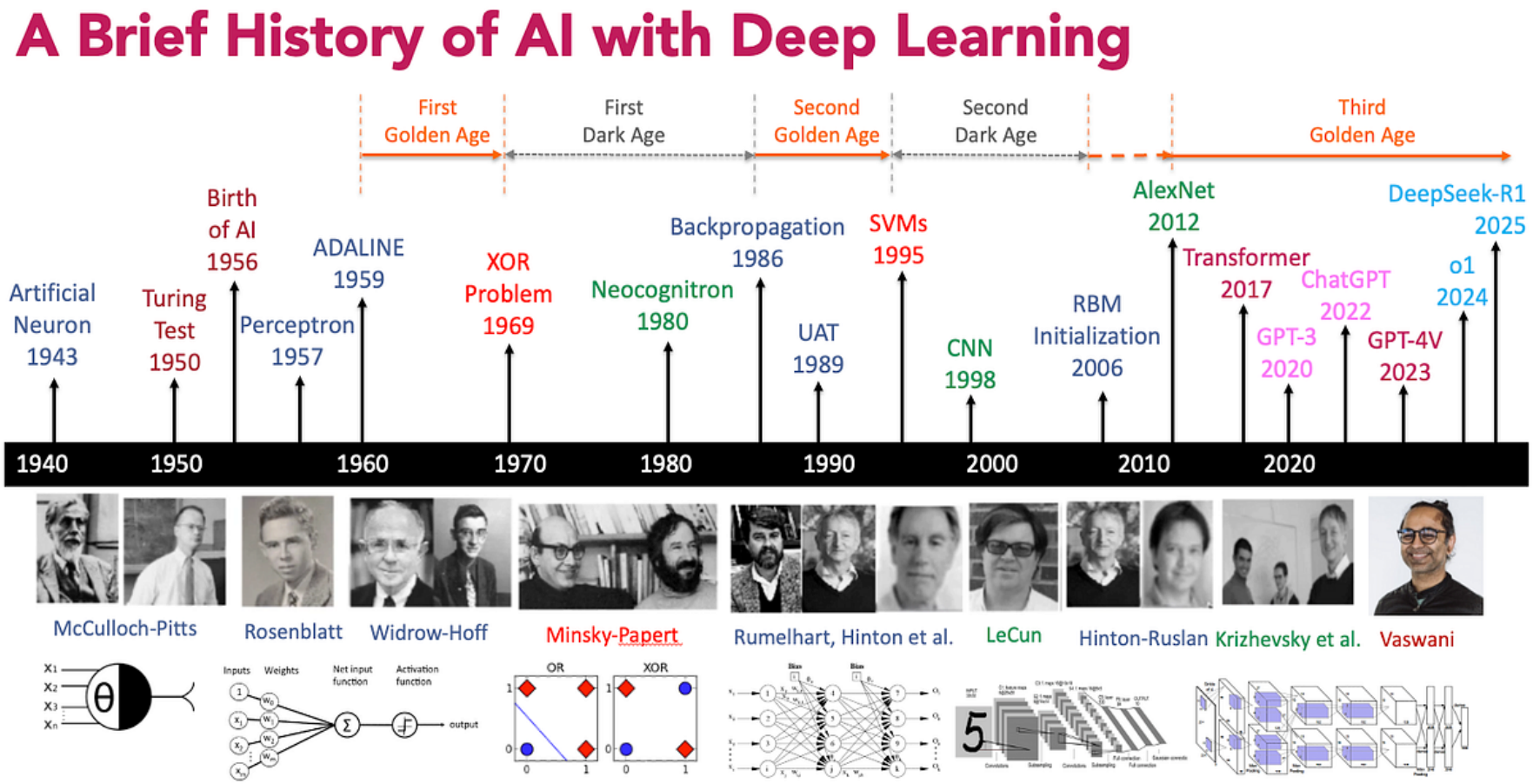
Project Guidelines

- To be discussed after the lecture

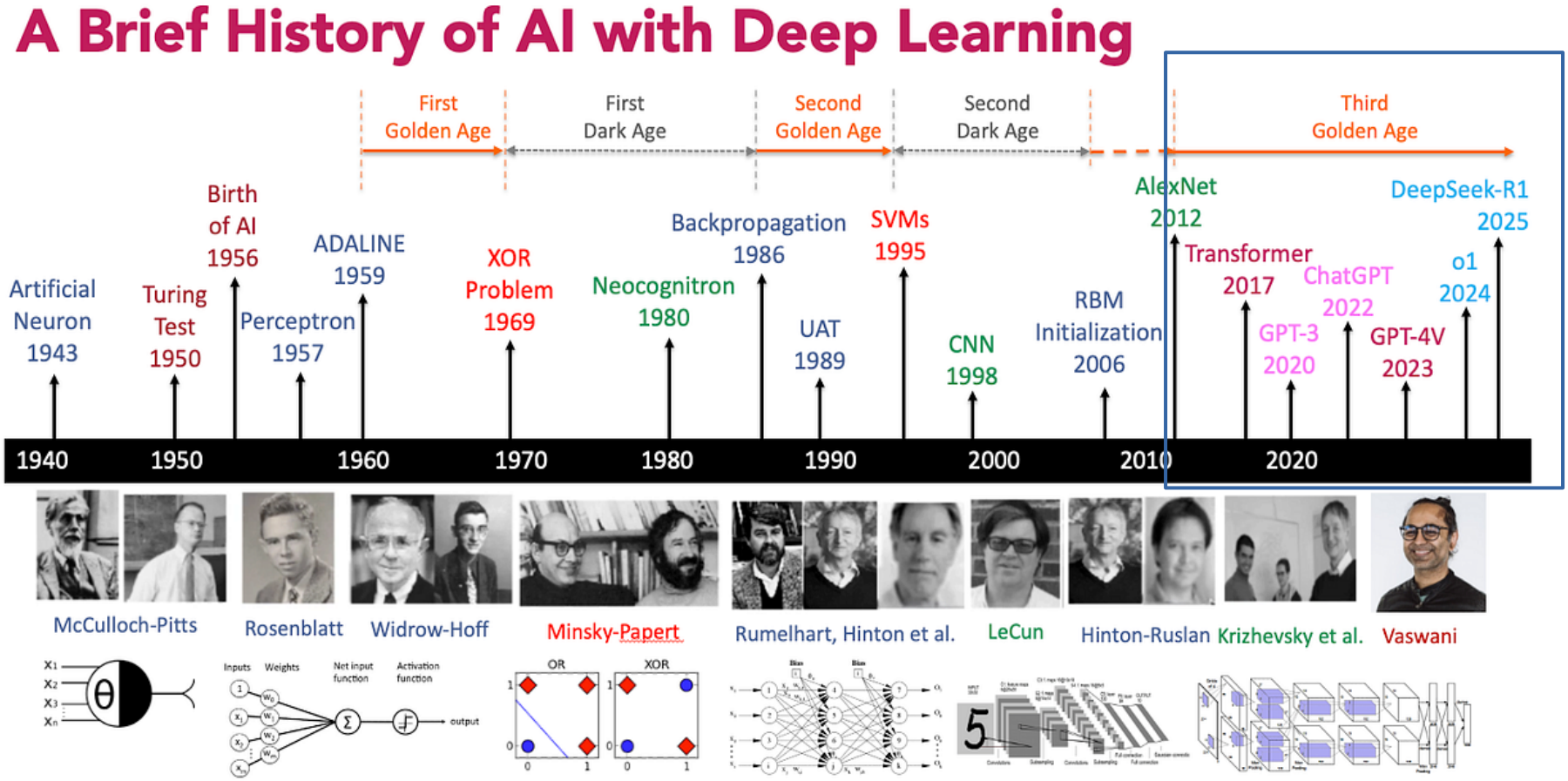
Introduction

Neural Networks – what is this class about?

What is this class about?



What is this class about?



What is this class about?

Several things affect the solutions that are obtained in a ML problem

Architectures **Learning / Training Algorithm**

Initialization **Data**

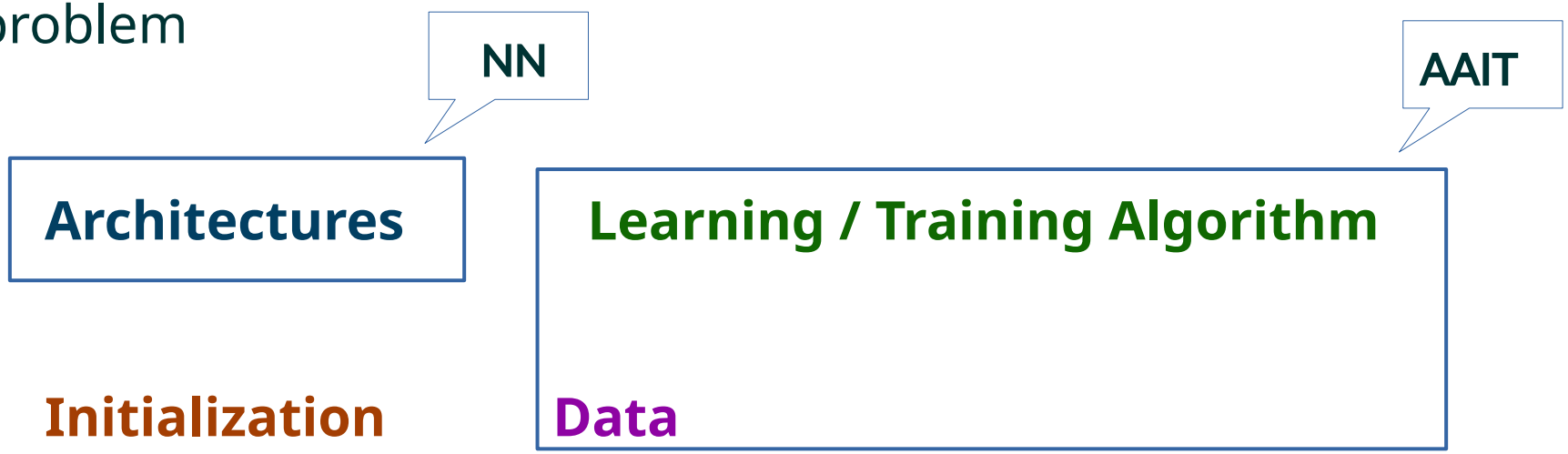
What is this class about?

Several things affect the solutions that are obtained in a ML problem



What is this class about?

Several things affect the solutions that are obtained in a ML problem



What is this class about? - **Neural Network Architectures**

Several things affect the solutions that are obtained in a ML problem

Architectures **Learning / Training Algorithm**

Initialization **Data**

- The **expressiveness** of a Neural Network **Architecture** defines the ***diversity*** of functions that ***can be represented*** by varying (adjusting, learning) its **parameters**

What is this class about? - **Neural Network Architectures**

- Understanding the **building blocks** needed to create neural network architectures
- Examining examples of architectures ***applied to*** different problem types
  - Image Classification / Captioning
  - Semantic / Instance Segmentation
  - Object Detection
  - Time series prediction / classification
  - Text Translation / Classification
  - Node / Relation classification / inference in structured knowledge

What is this class about? - **Neural Network Architectures**

- Understanding the **building blocks** needed to create neural network architectures
- Examining examples of architectures ***applied to*** different problem types

- Multi-Layered Perceptron (**Linear** Layers)
- **Convolutional** Neural Networks
- **Recurrent** Neural Networks
- Attention-based Models (aka **Transfomer** architectures)
- **Graph-based** Neural Networks

What is this class about? - **Neural Network Architectures**

- Understanding the **building blocks** needed to create neural network architectures
- Examining examples of architectures ***applied to*** different problem types

- Multi-Layered Perceptron (**Linear** Layers)
- **Convolutional** Neural Networks
- **Recurrent** Neural Networks
- Attention-based Models (aka **Transfomer** architectures)
- **Graph-based** Neural Networks

- Focus on **supervised learning problems**

Recap

Supervised Learning

Supervised Learning – the formulation

- **Dataset** with *n* exemples $\{x_i, y_i\}_{i=1}^{n}$
- Create a **decision model** (a **parameterized** function) that predicts $\hat{y}_i = f_\theta(x_i)$
- Define an **error function** $J(\hat{y}_i, y_i) \in \mathbb{R}$

Objective: find model parameters θ^* that minimize:

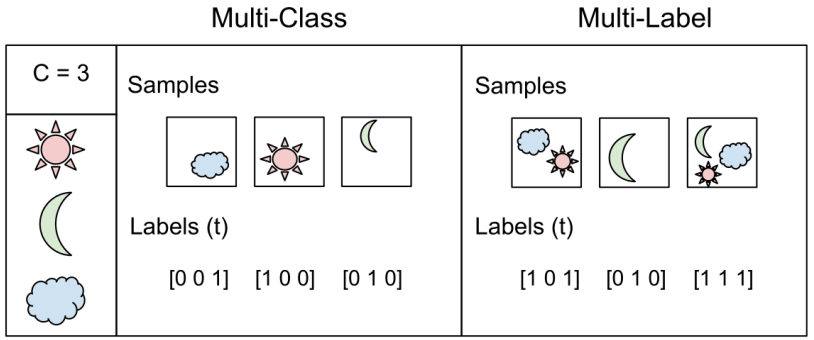
$$\theta^* = argmin_\theta \sum_{i=1}^{n} J(f_\theta(x_i), y_i)$$

Supervised Learning – Decision Models

- What is it we are predicting? (what is $\hat{y}$)
  - **Regression –** prediction is a **numerical value** / **sequence of numerical values** (if predicting over a window in time)
  - **Classification –** prediction is a **class label**
  - **Structured predictions**

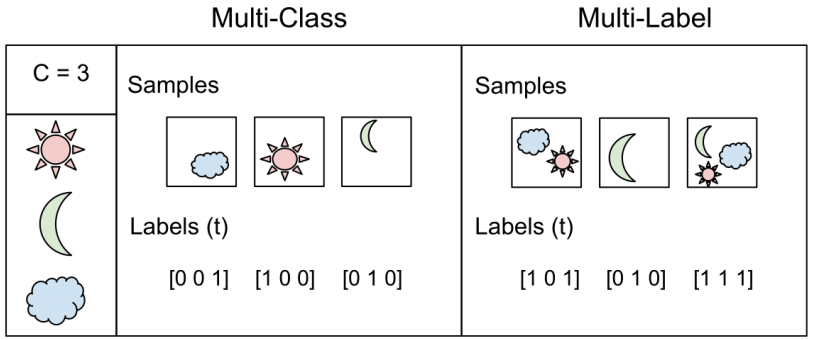
Supervised Learning – Decision Models

- What is it we are predicting? (what is $\hat{y}$)
  - **Regression –** prediction is a **numerical value** / **sequence of numerical values** (if predicting over a window in time)
  - **Classification –** prediction is a **class label**
  - **Structured predictions (e.g.):**
    - **Multi-label** classification



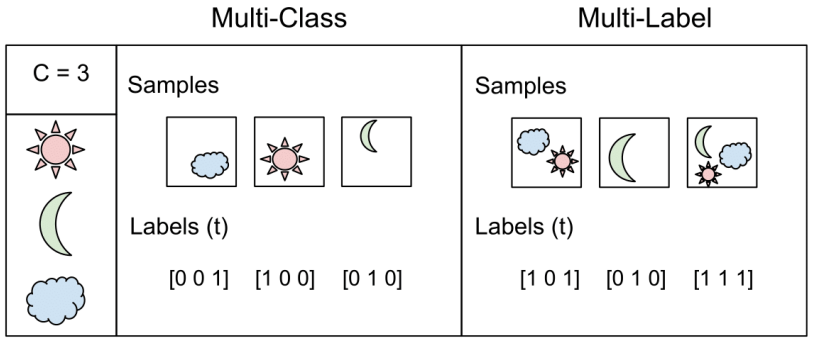
Supervised Learning – Decision Models

- What is it we are predicting? (what is $\hat{y}$)
  - **Regression –** prediction is a **numerical value** / **sequence of numerical values** (if predicting over a window in time)
  - **Classification –** prediction is a **class label**
  - **Structured predictions (e.g.):**
    - **Multi-label** classification
    - Image **Captions**



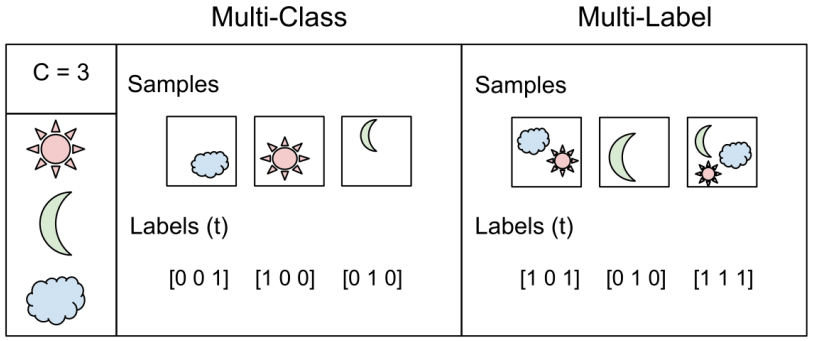
Supervised Learning – Decision Models

- What is it we are predicting? (what is $\hat{y}$)
  - **Regression –** prediction is a **numerical value** / **sequence of numerical values** (if predicting over a window in time)
  - **Classification –** prediction is a **class label**
  - **Structured predictions (e.g.):**
    - **Multi-label** classification
    - Image **Captions**
    - **Translation**



Supervised Learning – Decision Models

- What is it we are predicting? (what is $\hat{y}$)
  - **Regression –** prediction is a **numerical value** / **sequence of numerical values** (if predicting over a window in time)
  - **Classification –** prediction is a **class label**
  - **Structured predictions (e.g.):**
    - **Multi-label** classification
    - Image **Captions**
    - **Translation**
    - 2D / 3D **segmentation masks / bounding boxes**



Note: structured predictions can be decomposed into simpler regression and/or classification objectives – e.g. object detection = bounding box regression + object classification

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

How do we define $J(\hat{y}, y; \theta) \in \mathbb{R}$?

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

How do we define $J(\hat{y}, y; \theta) \in \mathbb{R}$?

- Regression (e.g.)
  - Mean Squared Error (MSE) $J(\hat{y}, y; \theta) = \frac{1}{n} \sum_{i=1..n} (y_i - \hat{y}_i)^2$

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

How do we define $J(\hat{y}, y; \theta) \in \mathbb{R}$?

- Regression (e.g.)
  - Mean Squared Error (MSE) $J(\hat{y}, y; \theta) = \frac{1}{n} \sum_{i=1..n} (y_i - \hat{y}_i)^2$
- Classification
  - It matters what ***kind*** of model we are using
    - Does the model learn a **decision boundary** (e.g. SVM, Decision Trees)
    - Is it a **model** of a **probability**?

$$\underbrace{p(y|x)}_{\text{discriminative}} \qquad \underbrace{p(y, x) = p(y|x) \cdot p(x)}_{\textit{generative}}$$

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

How do we define $J(\hat{y}, y; \theta) \in \mathbb{R}$?

- Regression (e.g.)
  - Mean Squared Error (MSE)

$$J(\hat{y}, y; \theta) = \frac{1}{n} \sum_{i=1..n} (y_i - \hat{y}_i)^2$$

- Classification
  - It matters what ***kind*** of model we are using
    - Does the model learn a **decision boundary** (e.g. SVM, Decision Trees)
    - Is it a **model** of a **probability**?

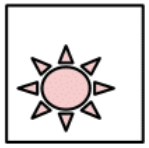
$$\underbrace{p(y|x)}_{\text{discriminative}} \qquad \underbrace{p(y, x) = p(y|x) \cdot p(x)}_{\textit{generative}}$$

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

- Classification where the model output is a **discriminative probability**

An example

$x_i =$ 

$$p(y_i|x_i) = \begin{bmatrix} p(y_i = \text{sun}|x_i) \\ p(y_i = \text{moon}|x_i) \\ p(y_i = \text{cloud}|x_i) \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.3 \\ 0.1 \end{bmatrix} = \hat{y}_i$$

$$\hat{y}_i = H_{\hat{y}_i} = -\sum_{c \in \text{sun}, \text{moon}, \text{cloud}} p(\hat{y}_i = c) \cdot \log\left(p(\hat{y}_i = c)\right)$$

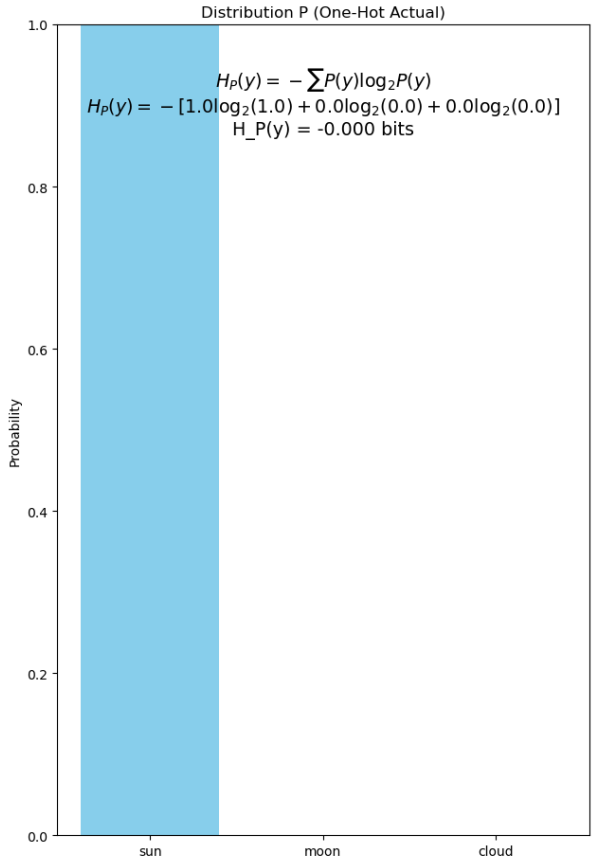
Entropy of $H_{\hat{y}_i} = -\left(0.6 \cdot \log(0.6) + 0.3 \cdot \log(0.3) + 0.1 \cdot \log(0.1)\right) \approx 1.295\, bits$

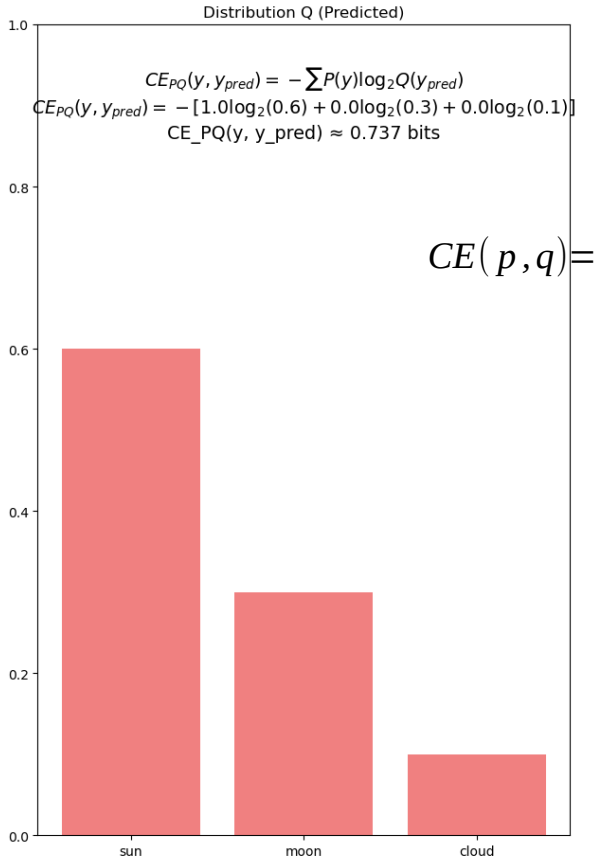
Supervised Learning – Error Functions

How do we **measure** an error in prediction?

- Classification where the model output is a **discriminative probability**

Cross Entropy (between two probability distributions **p** and **q**)





$$CE(p,q) = -\sum_{y} p(y) \cdot \log(q(y))$$

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

- Classification where the model output is a **discriminative probability**
  - **Cross Entropy** (between two probability distributions **p** and **q**)

$$CE(p,q) = -\sum_{y} p(y)\log(q(y))$$

  - **Cross Entropy** for multi-class classification with *C* classes

$$CE(y, f_\theta(x)) = -\sum_{j=1}^{C} y_j \log(\underbrace{f_\theta(x_j)}_{\hat{y}_j}) \qquad y \in \mathbb{R}^C = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \hat{y} \in \mathbb{R}^C = \begin{bmatrix} 0.6 \\ 0.3 \\ 0.1 \end{bmatrix}$$

  - **Binary Cross Entropy** (C=2)

$$CE(y, f_\theta(x)) = -y\log(\underbrace{f_\theta(x)}_{\hat{y}}) + (1-y)\log(1-\underbrace{f_\theta(x)}_{\hat{y}})$$

Supervised Learning – Error Functions

How do we **measure** an error in prediction?

- Classification where the model output **is turned into** a discriminative probability
  - **Multinomial Logistic Loss** = **Softmax Classifier +** Cross Entropy

    - **Softmax Classifier**

$$p(Y=k|X=x_i)=\frac{e^{s_k}}{\sum_j e^{s_j}} \qquad \underbrace{s_i=f_\theta(x_i)}_{\text{logit}}$$

    - **Cross Entropy over a Softmax Classifier**

$$CE(y, f_\theta(x))=-\log P(Y=y_i|X=x_i)=-\log\left(\frac{e^{s_i}}{\sum_j e^{s_j}}\right)$$

      - **Note:** in the C-class Cross Entropy sum, all the elements $y_j=0$ can be eliminated from the summation, leaving only the logit of the **target class**

Supervised Learning – how do we set up an **optimization objective**?

Supervised Learning – how do we set up an **optimization objective**?

- **Maximum Likelihood (MLE) – Negative Log Likelihood**
- **Kullback-Leibler Divergence - KL**

Supervised Learning – how do we set up an **optimization objective**? → MLE

Maximum Likelihood Estimation (MLE) - method of estimating the ***parameters*** of a model ***given observations***, by finding the parameter values that ***maximize the likelihood of making the observations given the parameters***

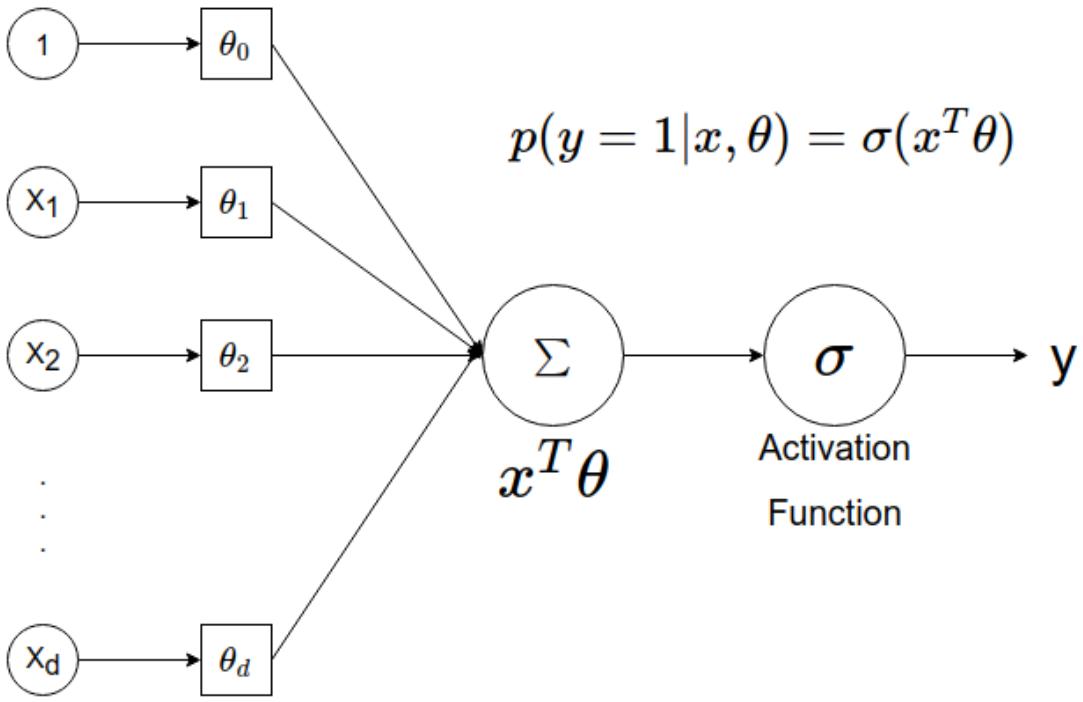
MLE property: for **i.i.d. data** *MLE minimizes* ***Cross-Entropy*** in multiclass classification

i.i.d = *independant* and **identically distributed *samples*** $\{x_i, y_i\}$ in the datasets

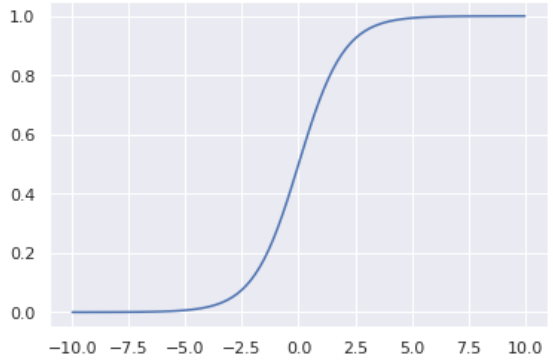
Supervised Learning – how do we set up an optimization **objective**? → MLE

MLE property: for i.i.d. data, *MLE minimizes Cross-Entropy*

Example for binary classification using **Logistic Regression**



$$p(y = 1|x, \theta) = \sigma(x^T\theta)$$



Logistic Sigmoid Function

$$\sigma(\eta) = \frac{1}{1+e^{-\eta}}$$

Supervised Learning – how do we set up an optimization **objective**? → MLE

MLE property: for i.i.d. data, *MLE minimizes Cross-Entropy*

Example for binary classification using **Logistic Regression**

- In Logistic Regression the target prediction p(y=1 | x, θ) behaves like a Bernoully Random Variable

$$p(x|\theta) = \begin{matrix} \theta, & if\ x=1 \\ 1-\theta, & if\ x=0 \end{matrix}$$

$$p(x|\theta) = Ber(x|\theta) = \theta^x (1-\theta)^{1-x}$$

Supervised Learning – how do we set up an **optimization objective**? → MLE

MLE property: for i.i.d. data, *MLE minimizes Cross-Entropy*

Example for binary classification using **Logistic Regression**

$$p(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{\theta}) = \prod_{i=1}^{n} p(y_i|\boldsymbol{x_i},\boldsymbol{\theta}) = \prod_{i=1}^{n} Ber(y_i|\sigma(\boldsymbol{x_i}\boldsymbol{\theta}))$$

$$= \prod_{i=1}^{n} \underbrace{\left[\frac{1}{1+e^{-x_i\theta}}\right]}_{\pi_i}^{y_i} \left[1-\frac{1}{1+e^{-x_i\theta}}\right]^{1-y_i}$$

Maximize ***likelihood*** $p(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{\theta})$ → *minimize* **negative log likelihood**

$$J(\boldsymbol{\theta}) = -\log P(\boldsymbol{y}|\boldsymbol{X},\boldsymbol{\theta}) = \underbrace{-\sum_{i=1}^{n} y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)}_{\textbf{cross-entropy}}$$

Supervised Learning – how do we define an **optimization objective - KL**

For Logistic Regression denote

$$\pi_i = p(y_i = 1 | x_i, \theta) \text{ and } 1 - \pi_i = p(y_i = 0 | x_i, \theta)$$

$D = \{(x_i, y_i), i = 1..n\}$ - is the *Empirical Data Distribution*

$D^m = \{(x_i, \pi_i), i = 1..n\}$ - is the modeled data distribution

In KL optimization we are trying to minimize the "discrepancy" between the *modeled* data distribution and the *empirical* one

Supervised Learning – how do we define an **optimization objective - KL**

Kullback-Leibler Divergence = measure of *difference* (not a proper distance) between two probability distributions

$$KL(P\|Q) = \sum_x P(x) \log\left(\frac{P(x)}{Q(x)}\right) = \underbrace{\sum_x P(x)\log(P(x))}_{-H(P(x))} \underbrace{- \sum_x P(x)\log(Q(x))}_{+H(P(x),Q(x))}$$

where $H(P(x),Q(x))$ is the $\boldsymbol{Cross-Entropy}$ measure of $P(x)$ and $Q(x)$ and $H(P(x))$ is the $\boldsymbol{Entropy}$ measure of $P(x)$

=> minimizing KL divergence = minimizing the cross-entropy

Supervised Learning – the formulation

- **Dataset** with *n* exemples $\{x_i, y_i\}_{i=1}^{n}$



- Create a **decision model** (a **parameterized** function) that predicts $\hat{y}_i = f_\theta(x_i)$



- Define an **error function** $J(\hat{y}_i, y_i) \in \mathbb{R}$



Objective: find model parameters θ^* that minimize:

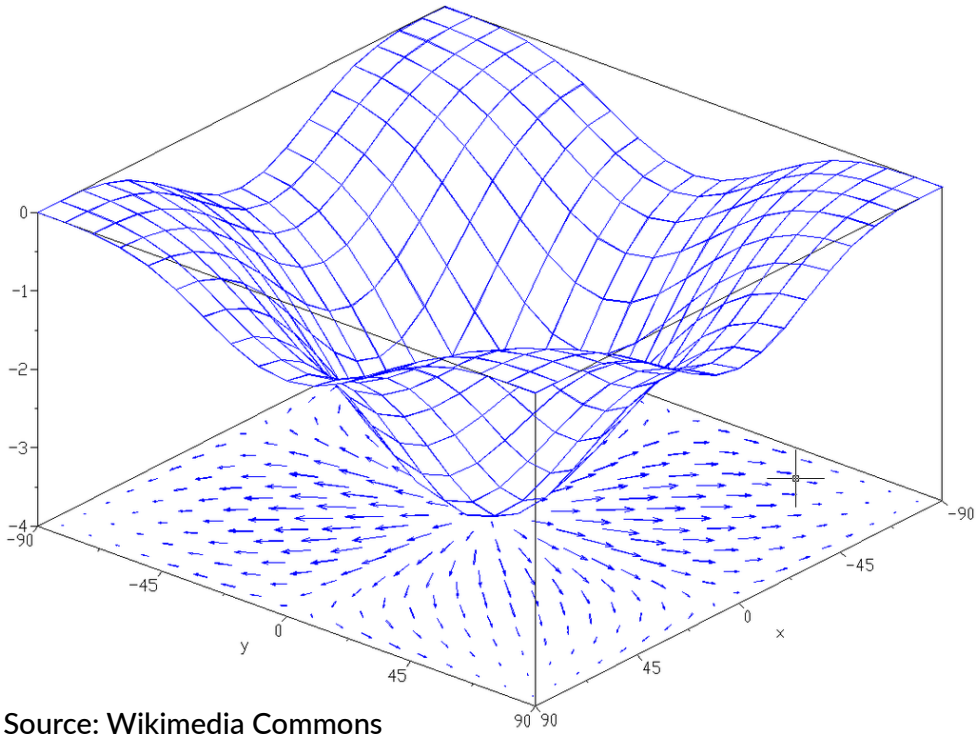
$$\theta^* = argmin_\theta \sum_{i=1}^{n} J(f_\theta(x_i), y_i)$$



Supervised Learning – how do we do the optimization?

Supervised Learning – how do we do the optimization?

Throughout this course we will most often resort to ***gradient-descent based optimization***



Source: Wikimedia Commons (https://bit.ly/2lOcCi9)

Gradient vector points in direction of ***steepest ascent =>*** for minimization take opposite direction

$$\theta^{t+1} = \theta^{t} - \lambda \frac{\partial J(\theta)}{\partial \theta}(\theta^{t})$$

where $\lambda \rightarrow$ learning rate

Recap

Backpropagation

Backpropagation

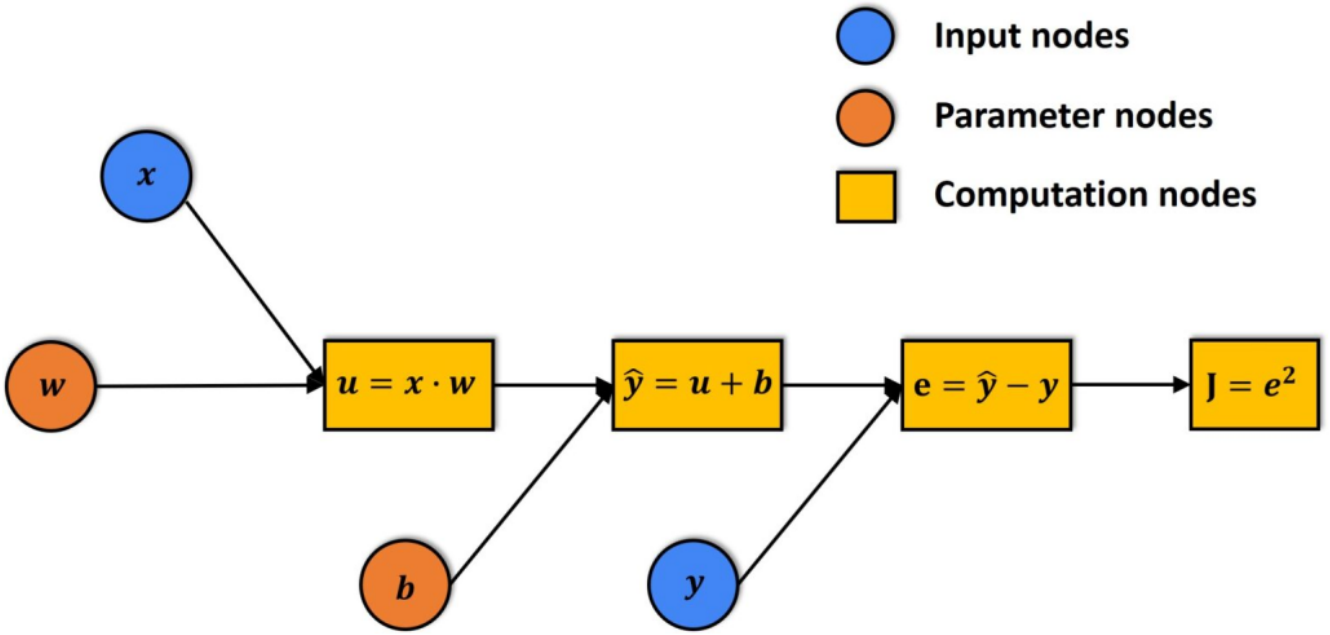
- Gradient descent computing **partial derivatives** of the error function $J(x,y;\theta)$ with respect to θ

- How do we compute such derivatives over arbitrary **Computation Graphs?**

Backpropagation – Computation Graphs

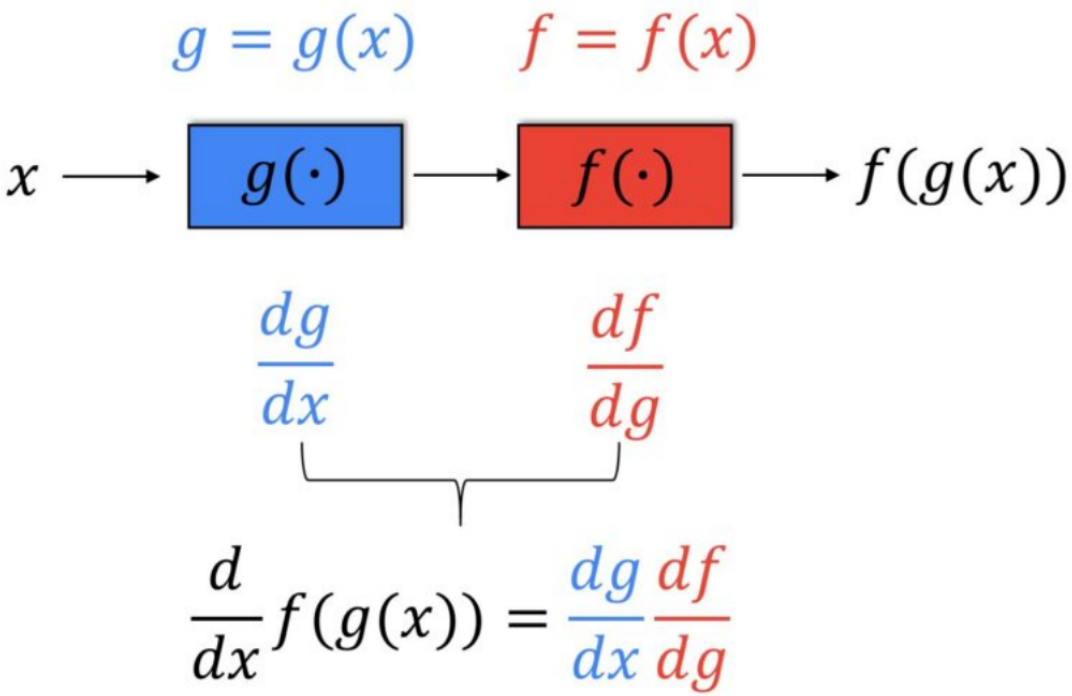
- How do we compute such derivatives over arbitrary **Computation Graphs?**
- Example for a Linear Regression

$$\hat{y} = x \cdot w + b; \mathcal{L} = (y - \hat{y})^2 = (y - x \cdot w - b)^2$$



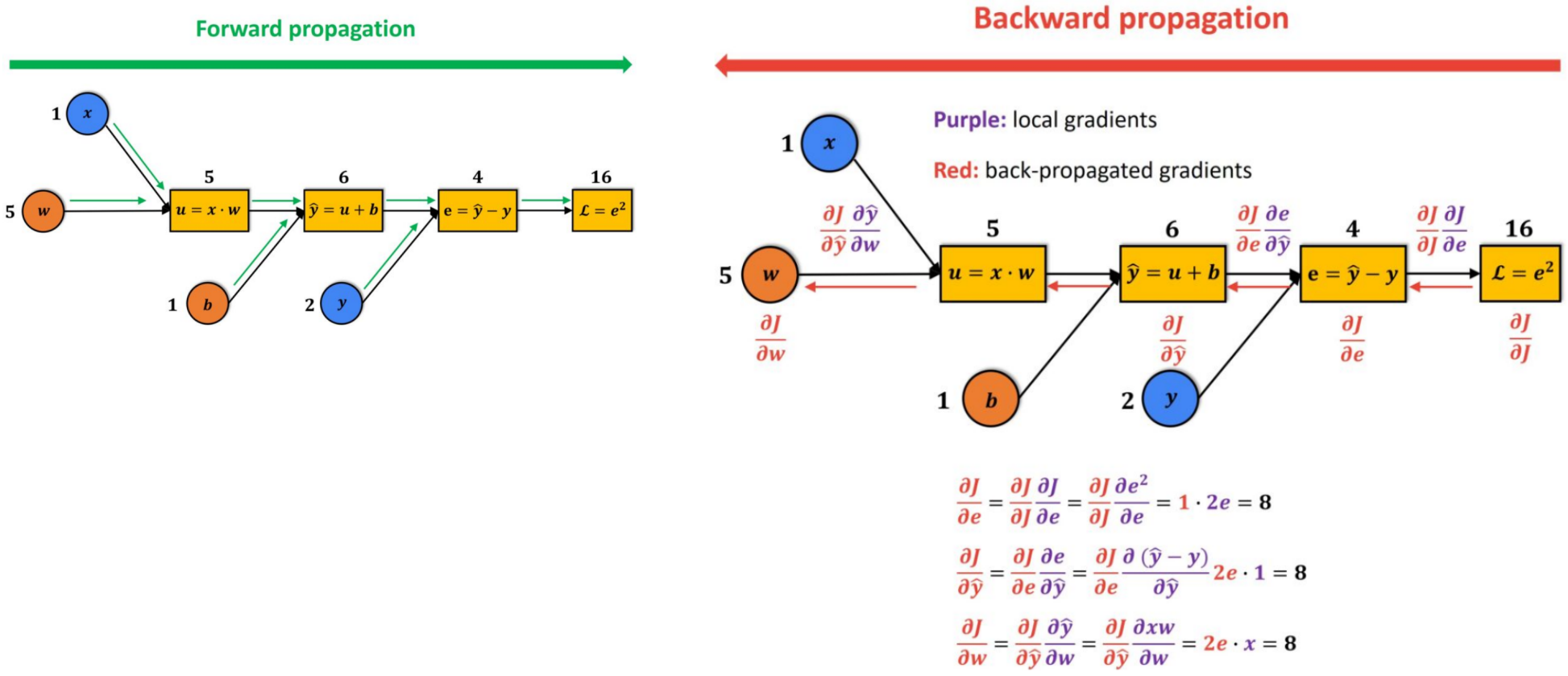
Backpropagation – Chain Rule of Derivation

- How do we compute such derivatives over arbitrary **Computation Graphs?**
- **Chain Rule of Derivation**



Backpropagation – Chain Rule of Derivation

- Linear Regression example



$$\frac{\partial J}{\partial e} = \frac{\partial J}{\partial J}\frac{\partial J}{\partial e} = \frac{\partial J}{\partial J}\frac{\partial e^2}{\partial e} = 1 \cdot 2e = 8$$

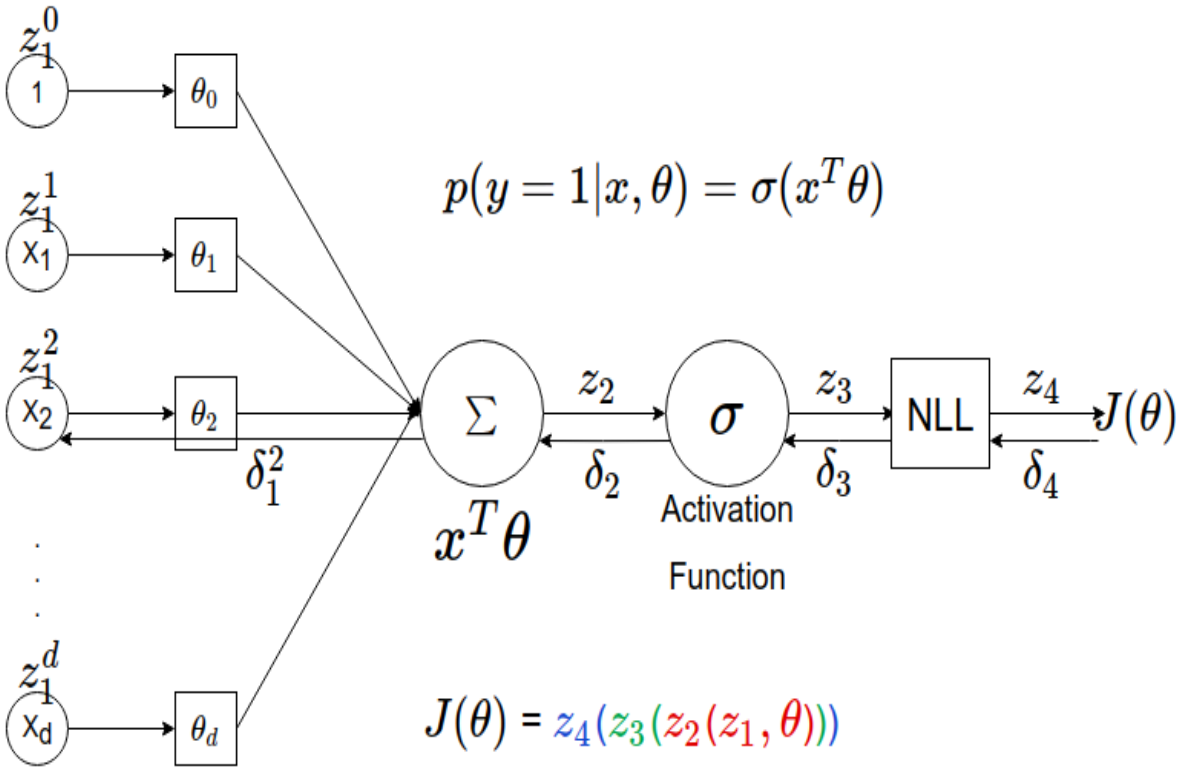
$$\frac{\partial J}{\partial \hat{y}} = \frac{\partial J}{\partial e}\frac{\partial e}{\partial \hat{y}} = \frac{\partial J}{\partial e}\frac{\partial (\hat{y} - y)}{\partial \hat{y}} 2e \cdot 1 = 8$$

$$\frac{\partial J}{\partial w} = \frac{\partial J}{\partial \hat{y}}\frac{\partial \hat{y}}{\partial w} = \frac{\partial J}{\partial \hat{y}}\frac{\partial xw}{\partial w} = 2e \cdot x = 8$$

Backpropagation – Chain Rule of Derivation

Backpropagation for a

"Logistic Regression Neural Net"



$$\delta_4 = J(\theta)$$

$$\delta_3 = \frac{\partial J}{\partial z_3} = \underbrace{\frac{\partial J}{\partial z_4}}_{1} \frac{\partial z_4}{\partial z_3}$$

$$\delta_3 = \frac{\partial -(y \log(z_3) + (1-y)\log(1-z_3))}{\partial z_3}$$

$$\delta_3 = \frac{-y}{z_3} - \frac{(1-y)}{(1-z_3)}$$

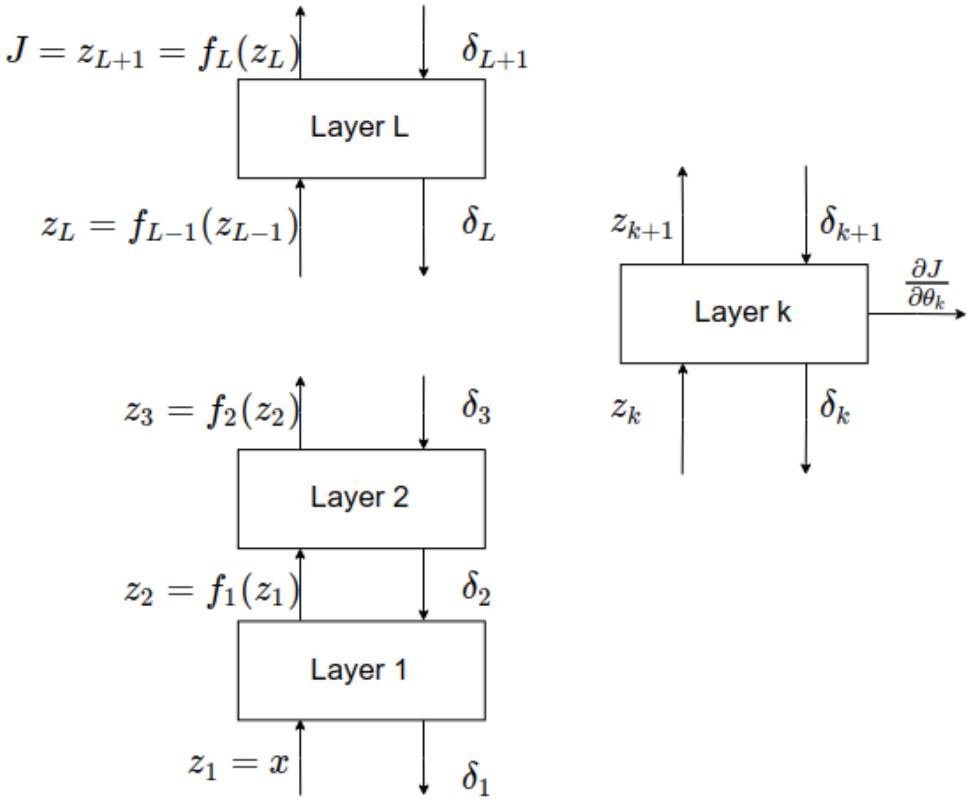
$$\delta_2 = \frac{\partial J}{\partial z_2} = \frac{\partial J}{\partial z_3}\frac{\partial z_3}{\partial z_2}$$

$$\delta_2 = \delta_3 \frac{\partial \sigma(z_2)}{\partial z_2} = \delta_3 \sigma(z_2)(1-\sigma(z_2))$$

$$\delta_1^2 = \frac{\partial J}{\partial z_1^2} = \delta_2 \frac{\partial \sum_{i=1}^{d} x_i \theta_i}{\partial x_2} = \delta_2 \theta_2$$

Backpropagation – Generalization to a Network Layer

In general



Forward pass:

$$z_{k+1} = f_k(z_k)$$

Backward pass:

$$\delta_k^i = \frac{\partial J}{\partial z_k^i} = \sum_j \frac{\partial J}{\partial z_{k+1}^j} \frac{\partial z_{k+1}^j}{\partial z_k^j} = \sum_j \delta_{k+1}^j \frac{\partial z_{k+1}^j}{\partial z_k^i}$$

$$\frac{\partial J}{\partial \theta_k} = \sum_j \frac{\partial J}{\partial z_{k+1}^j} \frac{\partial z_{k+1}^j}{\partial \theta_k} = \sum_j \delta_{k+1}^j \frac{\partial z_{k+1}^j}{\partial \theta_k}$$

Summary

- Take aways:
  - Class will focus on **neural network architectures** and **supervised learning setups**
  - Defining a prediction problem involves:
    - Having / creating a dataset
    - Defining a **parameterized model** (usually one that output discriminative probabilities
    - Defining an **error function**
    - Setting an **optimization objective** (usually MLE, sometimes KL)
  - **Optimization** usually carried out using variations of **Gradient Descent**
  - Gradient descent obtains gradient values through the **Backpropagation Algorithm**
    - Computation Graph
    - Chain rule of probability
    - Gradient based update rule